Experiment No. 6

Title: Develop a Node.js Application using Express.js and Mongoose to Connect with MongoDB

Aim:

To build a modular backend web server using Node.js, the Express.js framework, and Mongoose ODM, establishing a persistent database connection with MongoDB and exposing structured RESTful endpoints.

Tools Used:

- Node.js runtime v22 (ES Modules)
- Express.js v4.21.1 web framework
- Mongoose v8.7.1 ODM
- MongoDB Atlas cloud database
- Helmet, CORS, Morgan, Express JSON body parsing middleware
- JSON Web Token (jsonwebtoken) & bcryptjs for security

Methodology:

Express App Initialization → Middleware Pipeline Mounting → Mongoose DB Connection → Schema & Model Compilation → Controller Business Logic → Route Mounting → Client Integration

The backend server is structured following MVC design patterns. Express initializes HTTP server listeners and mounts a robust middleware chain (Helmet security headers, CORS, body parsers, logging). Mongoose connects to the MongoDB cluster via connection pooling. Schemas define document shapes, types, and validation constraints. Controllers encapsulate business logic and database queries, which routes map to HTTP verbs.

Theory:

Express.js is a minimalist, flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies server creation through a modular middleware pipeline where incoming HTTP requests pass through sequential functions capable of inspecting, modifying, or terminating requests.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, translates between JavaScript objects and BSON representations, and exposes rich query builders. Mongoose schemas define the fields, data types, default values, validators, and indexing strategies for collections.

In a 3-tier web architecture, Express functions as the Application/Logic Tier mediating between the Presentation Tier (React client) and the Data Tier (MongoDB). Security middleware like Helmet sets HTTP response headers, CORS manages cross-origin resource sharing, and JWT authentication middleware guards sensitive business endpoints.

Case Study Context:

Kitab Ghar, Badlapur — backend web service. The Express application (server/server.js) hosts modular routers for book inventory, reader accounts, subscriptions, rental transactions, and notifications, connected via Mongoose to the MongoDB Atlas cluster.

This experiment documents the genuine production backend architecture in server/, analyzing server initialization, middleware pipelines, Mongoose schema compilation, controllers, and authenticated endpoint delivery.

Functions / Methods / Technologies Used:

- express() and app.listen() for server lifecycle management
- app.use() for middleware composition (express.json, cors, helmet, morgan)
- mongoose.connect() with connection string and reconnection handling
- new mongoose.Schema() for data structure definition
- mongoose.model() for model compilation
- express.Router() for modular endpoint mapping
- jwt.sign() and jwt.verify() for token-based security

Procedure:

The Express service was started with its Mongoose connection, then health, catalogue, plan, authentication, and protected dashboard flows were exercised. Public and protected endpoints were compared so the middleware and controller behaviour could be observed separately.

1. Navigate to server/ and review package.json dependencies (express, mongoose, dotenv, cors, helmet).
2. Examine server/server.js to analyze middleware pipeline: helmet(), cors(), express.json(), morgan().
3. Inspect server/config/db.js to examine the Mongoose connection manager and error event handlers.
4. Review server/models/Book.js to study Mongoose Schema definition, types, and validation rules.
5. Review server/controllers/bookController.js to analyze async controller functions (getBooks, getBookById).
6. Review server/routes/bookRoutes.js to study how endpoints are bound to controller functions.
7. Review server/middleware/auth.js to trace JWT verification and role-based access control.
8. Start the backend server using 'npm run dev' inside server/; verify terminal confirms database connection.
9. Query GET http://localhost:5000/api/health; verify { status: 'ok', database: 'mongodb' }.
10. Query GET http://localhost:5000/api/books; verify JSON array of catalogue documents is returned.
11. Capture screenshots of server startup, API responses, and client frontend integration.

Code:

Code Block 6.1: Express Server Setup & Middleware Pipeline

File: `server/server.js` (Lines 1–32) — Initializes Express, configures security middleware, connects database, and mounts routes.

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import morgan from "morgan";
import connectDB from "./config/db.js";
import bookRoutes from "./routes/bookRoutes.js";
import authRoutes from "./routes/authRoutes.js";

dotenv.config();
connectDB();

const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_ORIGIN || "http://localhost:5173" }));
app.use(express.json());
app.use(morgan("dev"));

app.get("/api/health", (req, res) => {
  res.json({ status: "ok", service: "kitabghar-api", database: "mongodb" });
});

app.use("/api/books", bookRoutes);
app.use("/api/auth", authRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Code Block 6.2: Mongoose Schema Definition: Book Model

File: `server/models/Book.js` (Lines 1–24) — Defines schema fields, data types, validation rules, and indexes for books collection.

```
import mongoose from "mongoose";

const bookSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    author: { type: String, required: true, trim: true },
    description: { type: String, required: true },
    category: { type: String, required: true },
    coverImage: { type: String, default: "" },
    totalCopies: { type: Number, required: true, min: 1, default: 1 },
    availableCopies: { type: Number, required: true, min: 0, default: 1 },
    language: { type: String, default: "English" },
    status: { type: String, enum: ["active", "archived"], default: "active" }
  },
  { timestamps: true }
);

bookSchema.index({ title: "text", author: "text", category: "text" });

export const Book = mongoose.model("Book", bookSchema);
```

Output:

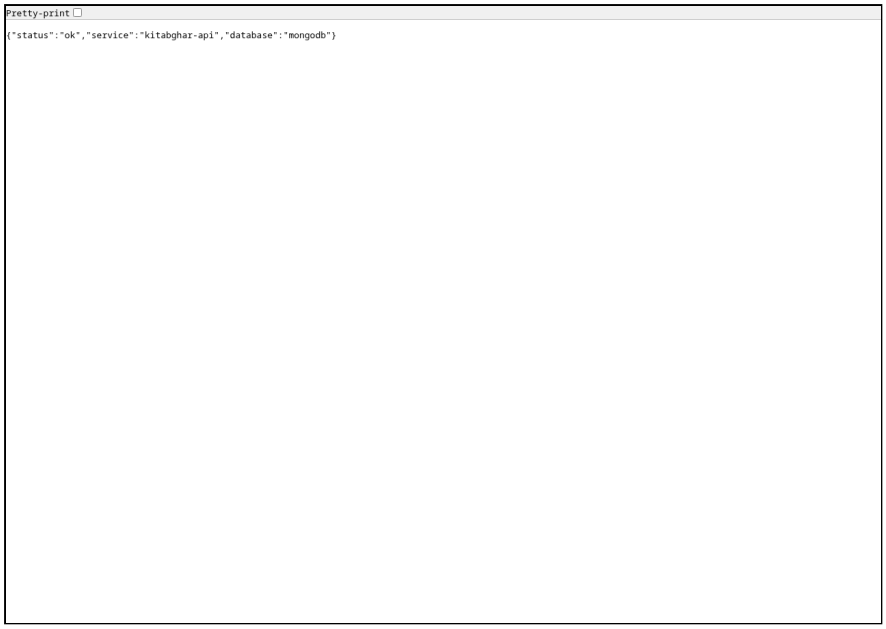


Figure 6.1: Express Server Health Check Endpoint (GET /api/health)

This screenshot shows the JSON response returned by GET /api/health. The endpoint verifies that the Express web service is listening, responsive, and connected to the MongoDB Atlas cluster.



Figure 6.2: REST API Books Catalogue Endpoint (GET /api/books)

This screenshot displays the raw JSON output from the /api/books endpoint. The Express controller queried MongoDB via Mongoose, returning 77 book documents with pagination metadata.

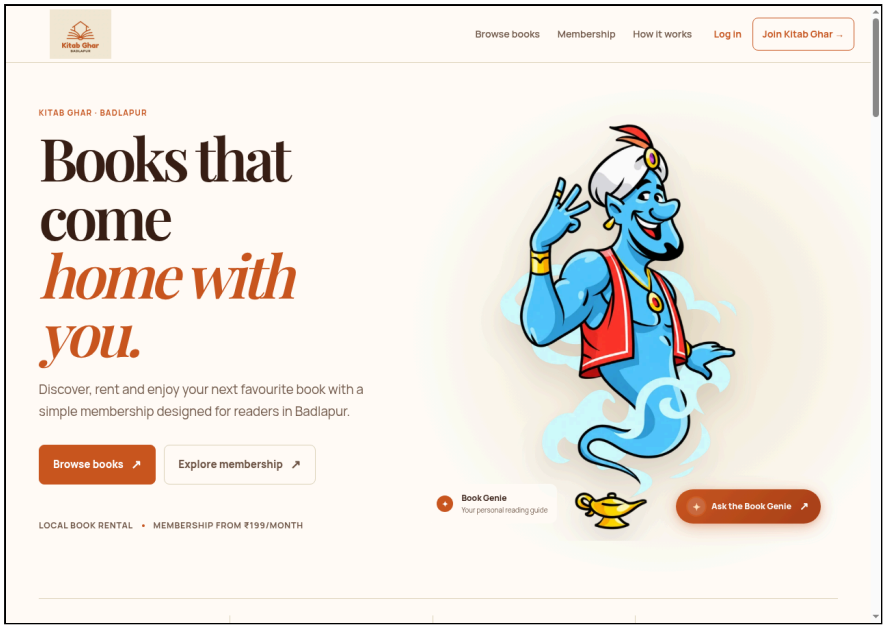


Figure 6.3: Kitab Ghar Main Frontend Connected to Express API

This screenshot shows the React frontend consuming live data from the Express backend via the Fetch API, verifying end-to-end communication between client and server. The books catalogue displayed on the frontend exactly matches the data stored in MongoDB, confirming that the Express middleware chain (CORS, body-parser, Helmet) correctly handles cross-origin requests from the React client.

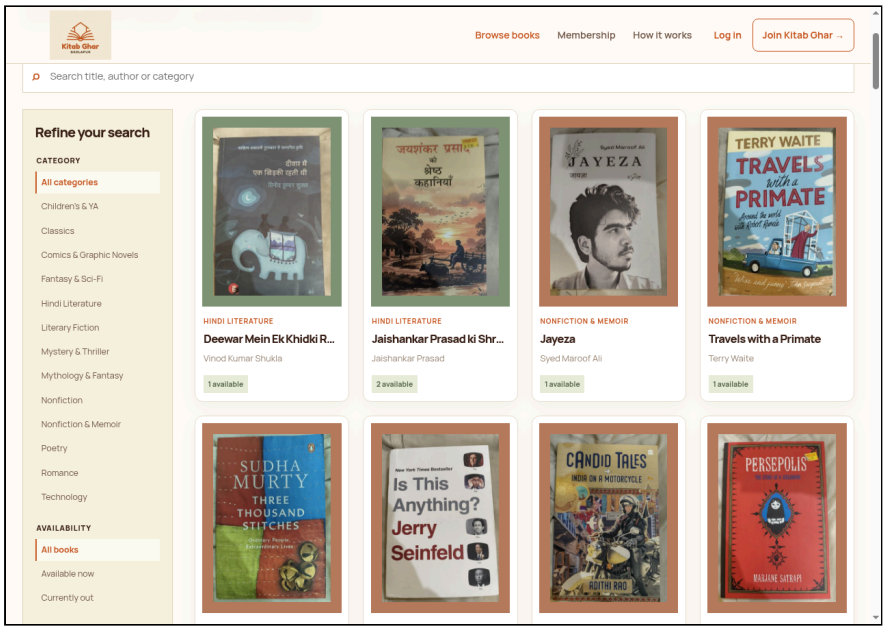


Figure 6.4: Books Catalogue Rendered from MongoDB Documents

This screenshot shows the live books catalogue rendered from MongoDB documents fetched through the Express API route /api/books, proving database-backed front-end presentation. Each BookCard component displays title, author, category badge, and real-time availability count sourced directly from the Mongoose query results, demonstrating the complete data flow from database to browser.

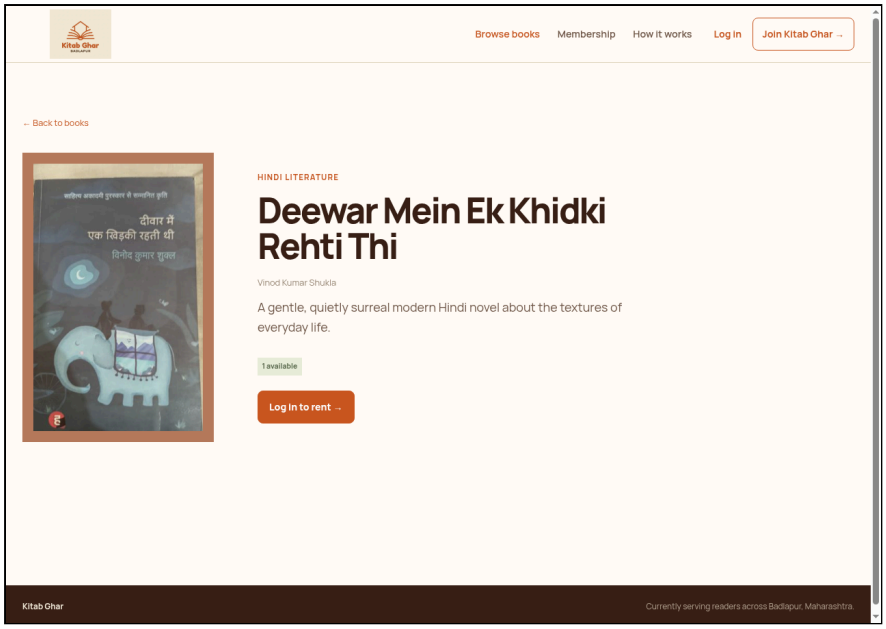


Figure 6.5: Dynamic Book Details View Backed by API Endpoint

This screenshot shows the book details view populated via the GET /api/books/:id Express route, demonstrating single-document retrieval through Mongoose's findById() query. The Express controller extracts the :id parameter from req.params, passes it to Mongoose, and returns the complete book document including description, rating, rental pricing, and availability status.

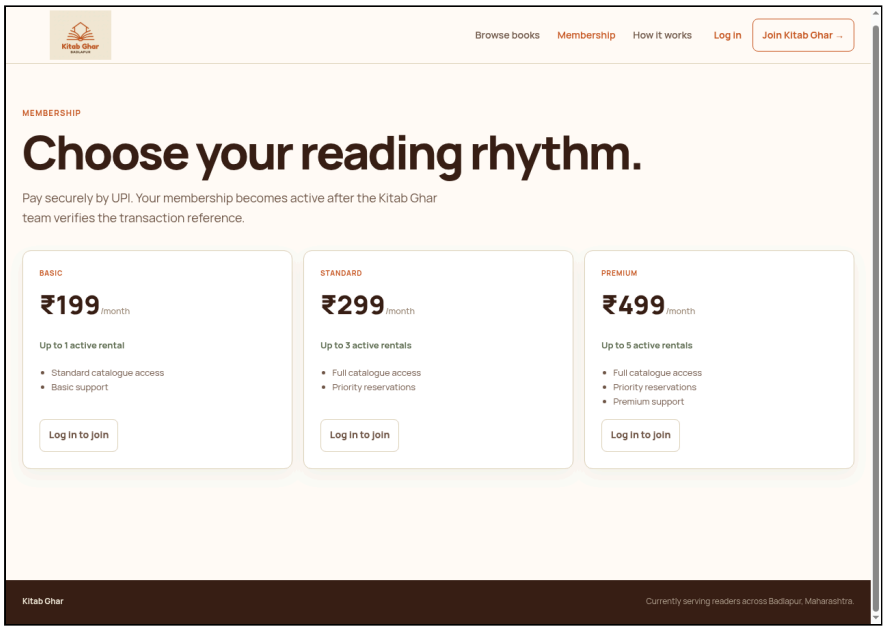


Figure 6.6: Membership Plans API Endpoint & Subscription Service

This screenshot shows membership plan options (Reader, Scholar, Patron tiers) delivered dynamically by the Express server's planController via the GET /api/plans endpoint, illustrating modular endpoint routing. Each plan document includes pricing, features, and duration fields stored in MongoDB and served through the Express-Mongoose pipeline.

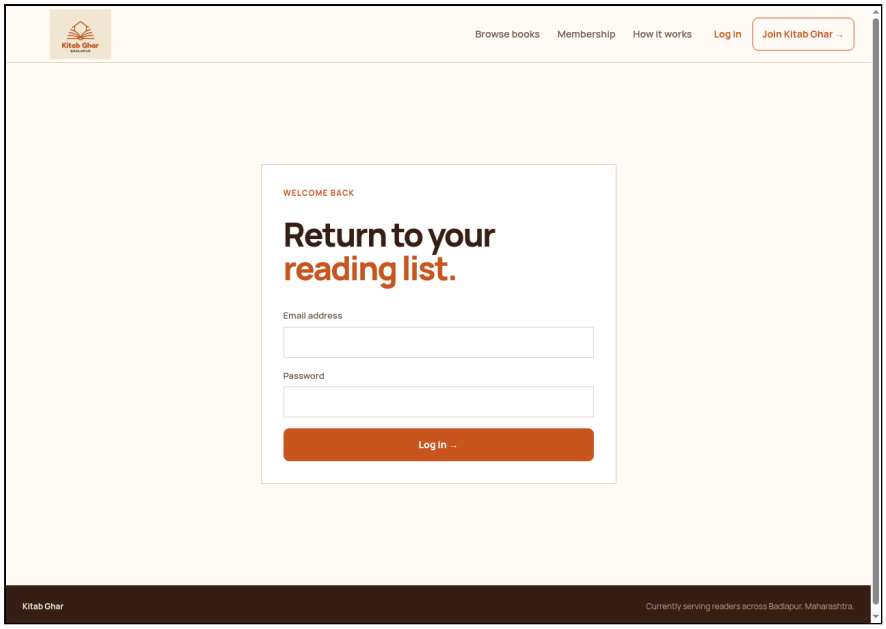


Figure 6.7: JWT Authentication & User Login (POST /api/auth/login)

This screenshot shows reader authentication. Submitting credentials to the POST /api/auth/login endpoint invokes bcrypt.compare() for secure password verification against the hashed value stored in MongoDB. Upon successful authentication, the Express controller issues a signed JSON Web Token (JWT) using jsonwebtoken.sign(), which the React client stores in localStorage for subsequent authenticated API requests.

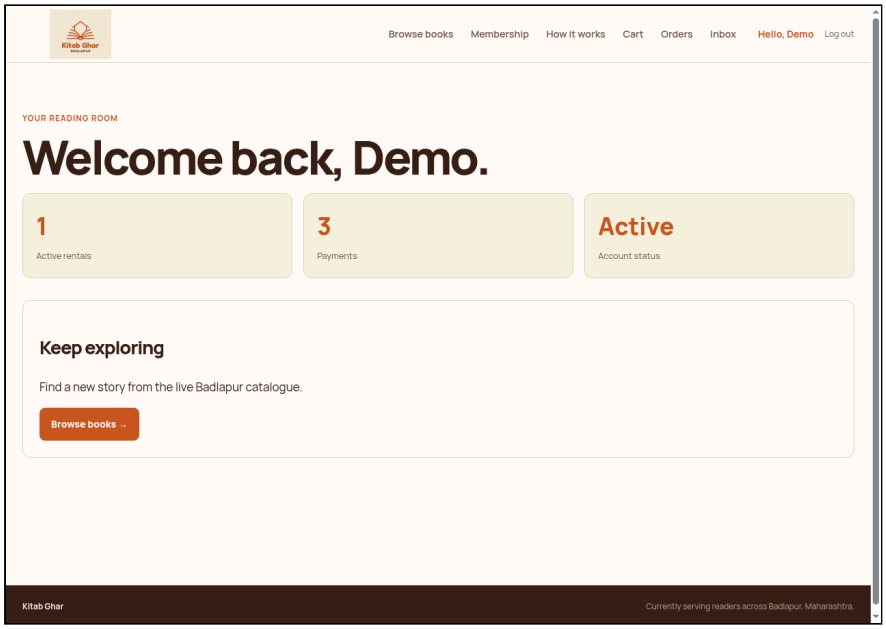


Figure 6.8: Protected Reader Dashboard Populated via Authenticated API

This screenshot displays the protected reader dashboard populated with personalized rental data. API requests from the React client include the JWT Bearer token in the Authorization header, allowing the Express auth middleware to verify the token, decode the user payload, and attach it to req.user. The controller then queries MongoDB for the authenticated user's active rentals, return dates, and membership status.

Result:

A complete backend web application was built using Node.js, Express.js, and Mongoose connected to MongoDB. The server establishes persistent database connectivity, validates document schemas, and exposes robust public and protected RESTful API endpoints.

The API returned live JSON data for public resources, while authentication issued a token and protected requests used that session to return reader-specific data. Schema, controller, and middleware responsibilities were visible across the observed requests.

Conclusion:

The experiment confirmed that Express.js and Mongoose form an efficient, expressive backend foundation for Node.js web development. The middleware architecture provides clear separation between security, routing, and business logic.

Mongoose models provide strong data validation and schema integrity while retaining the schema flexibility of MongoDB, ensuring a secure and scalable backend service.

This backend architecture demonstrates how Express routing, Mongoose models, and JWT middleware combine to deliver a secure, database-backed API.